

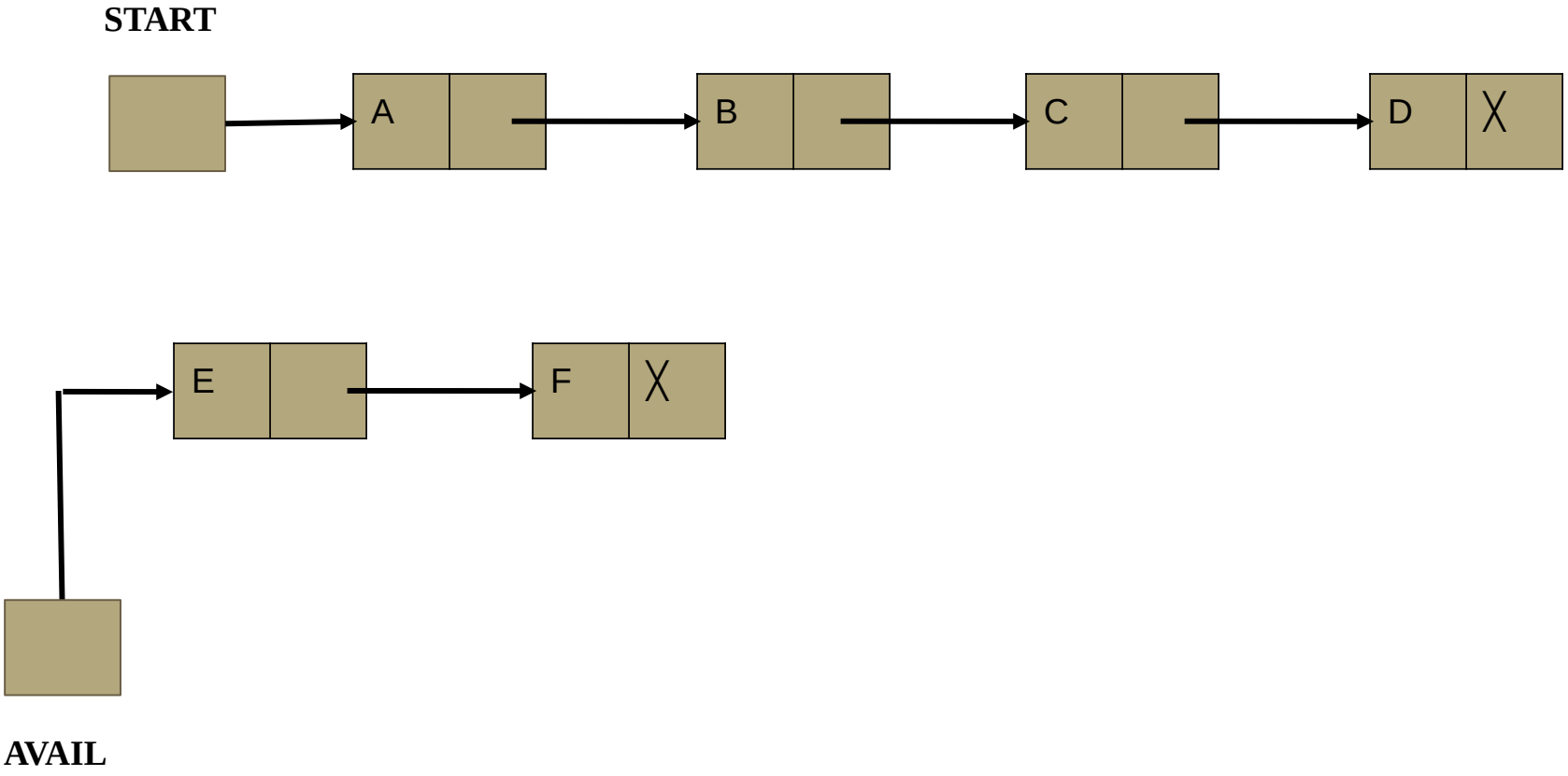
Data Structure(CSL202)

By-
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)



Insertion in a Linked List

Insertion at the Beginning in a Linked List



Insertion at the Beginning in a Linked List

1. **Check for available space (Overflow):**

   If AVAIL = NULL, print OVERFLOW and exit.

2. **Get a new node from AVAIL list:**

a. Set NEW ← AVAIL.

b. Set AVAIL ← LINK[AVAIL].

1. **Store ITEM in new node:**

   Set INFO[NEW] ← ITEM.

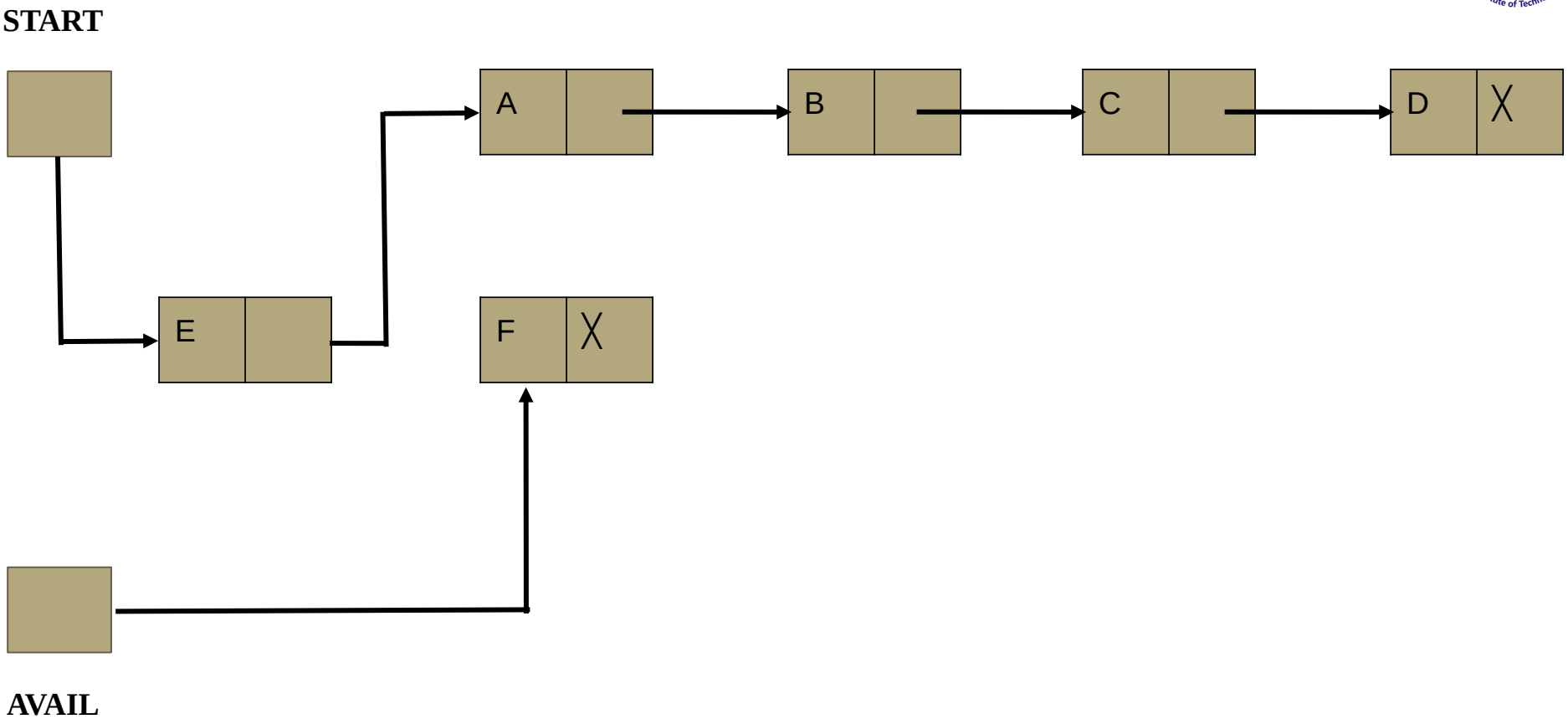
2. **Link new node to current first node:**

   Set LINK[NEW] ← START.

3. **Make new node the first node:**

   Set START ← NEW.

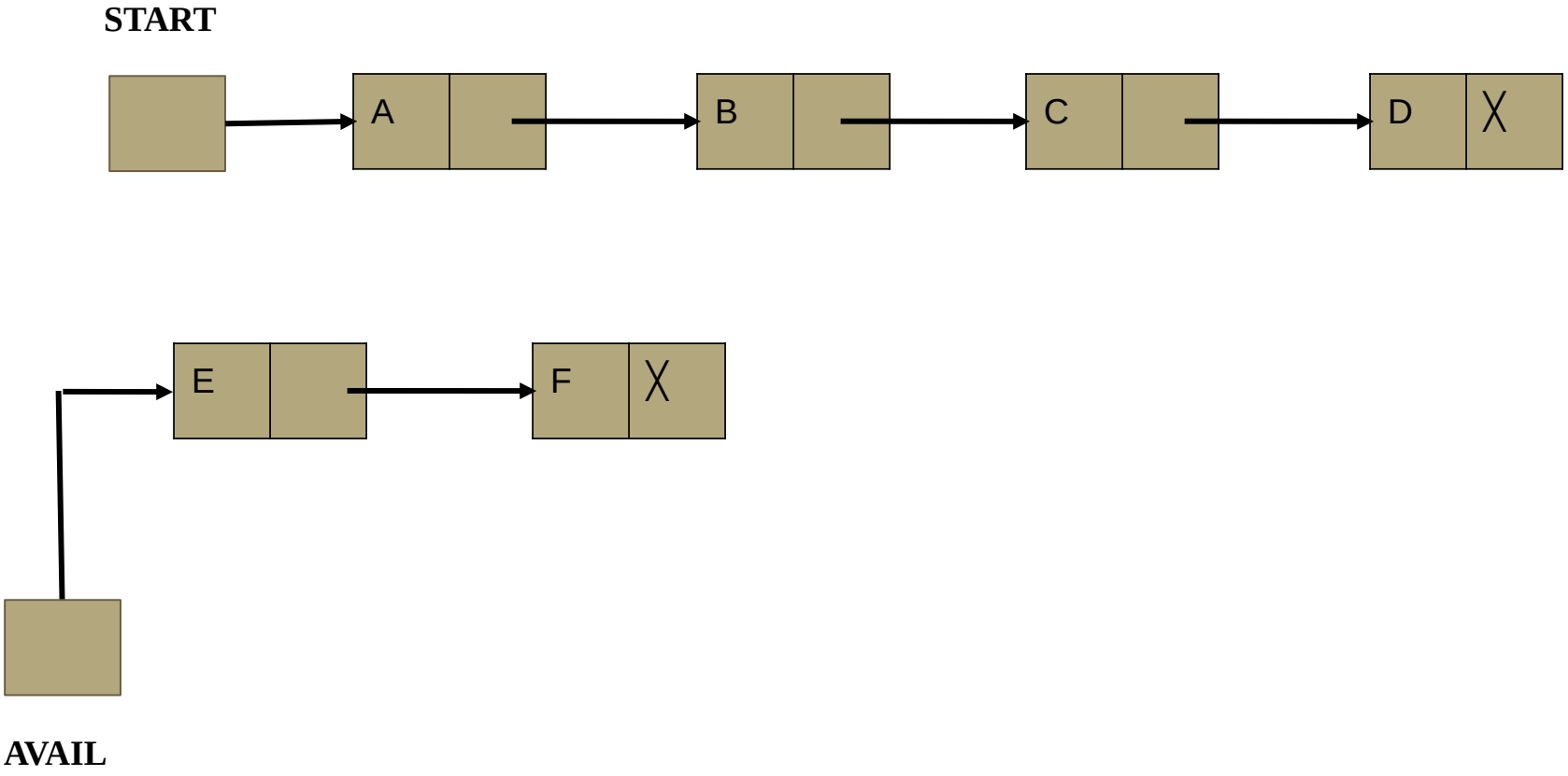
Insertion at the Beginning in a Linked List



Insertion at the Beginning in a Linked List

- Insertion at the start requires only constant-time pointer updates without traversal.
  - **Worst-** O(1)
  - **Average:** O(1)
  - **Best:** O(1)

Insertion at the End in a Linked Lis



Insertion at the End in a Linked List

1. **[Overflow?]**

   If AVAIL = NULL, print OVERFLOW and exit.

2. **[Remove first node from AVAIL list.]**

   Set NEW ← AVAIL

   Set AVAIL ← LINK[AVAIL]

3. **[Store ITEM in new node.]**

   Set INFO[NEW] ← ITEM

   Set LINK[NEW] ← NULL

4. **[Traverse to last node.]**

   Set PTR ← START

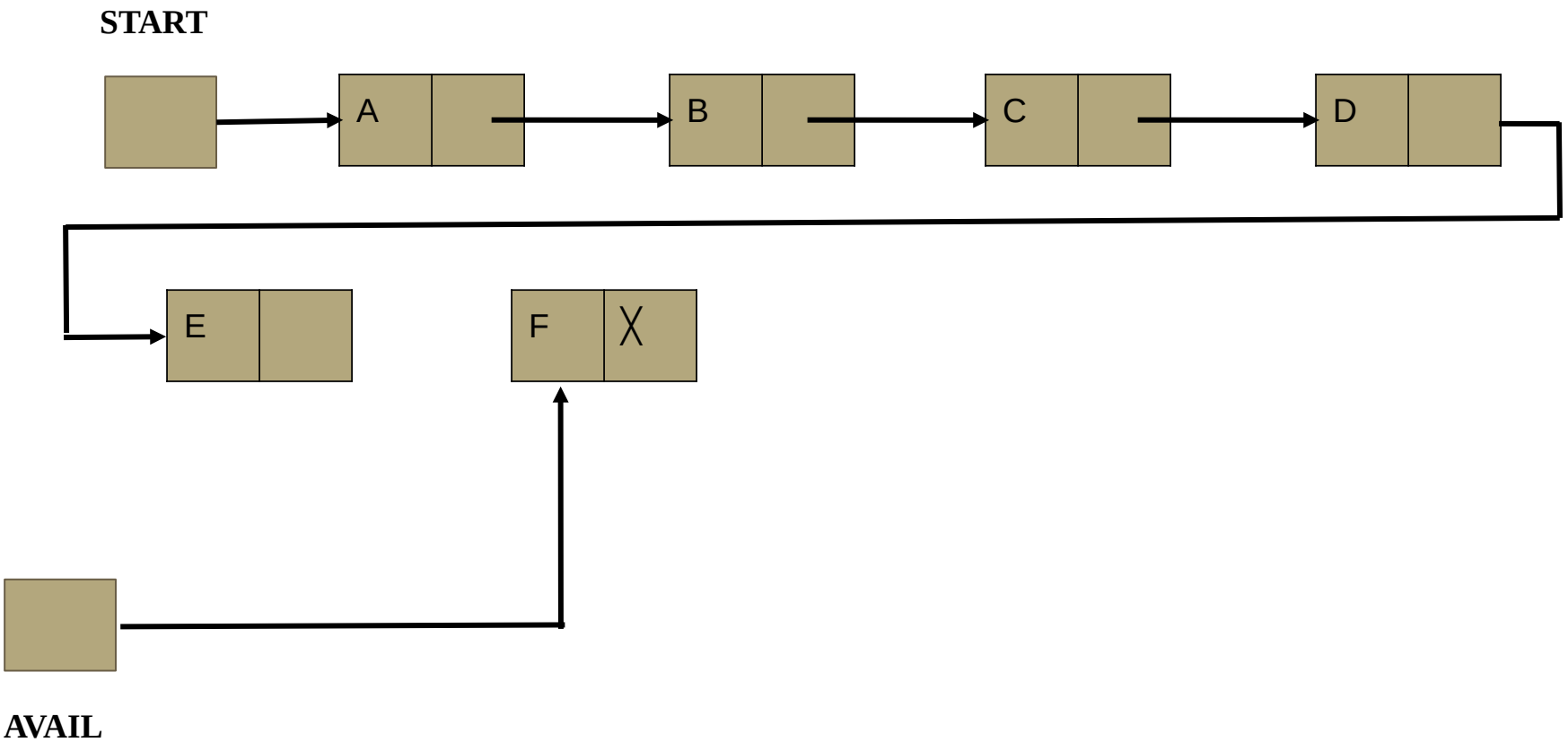
   While LINK[PTR] ≠ NULL:

   Set PTR ← LINK[PTR]

5. **[Insert new node at end.]**

   Set LINK[PTR] ← NEW

Insertion at the End in a Linked Lis

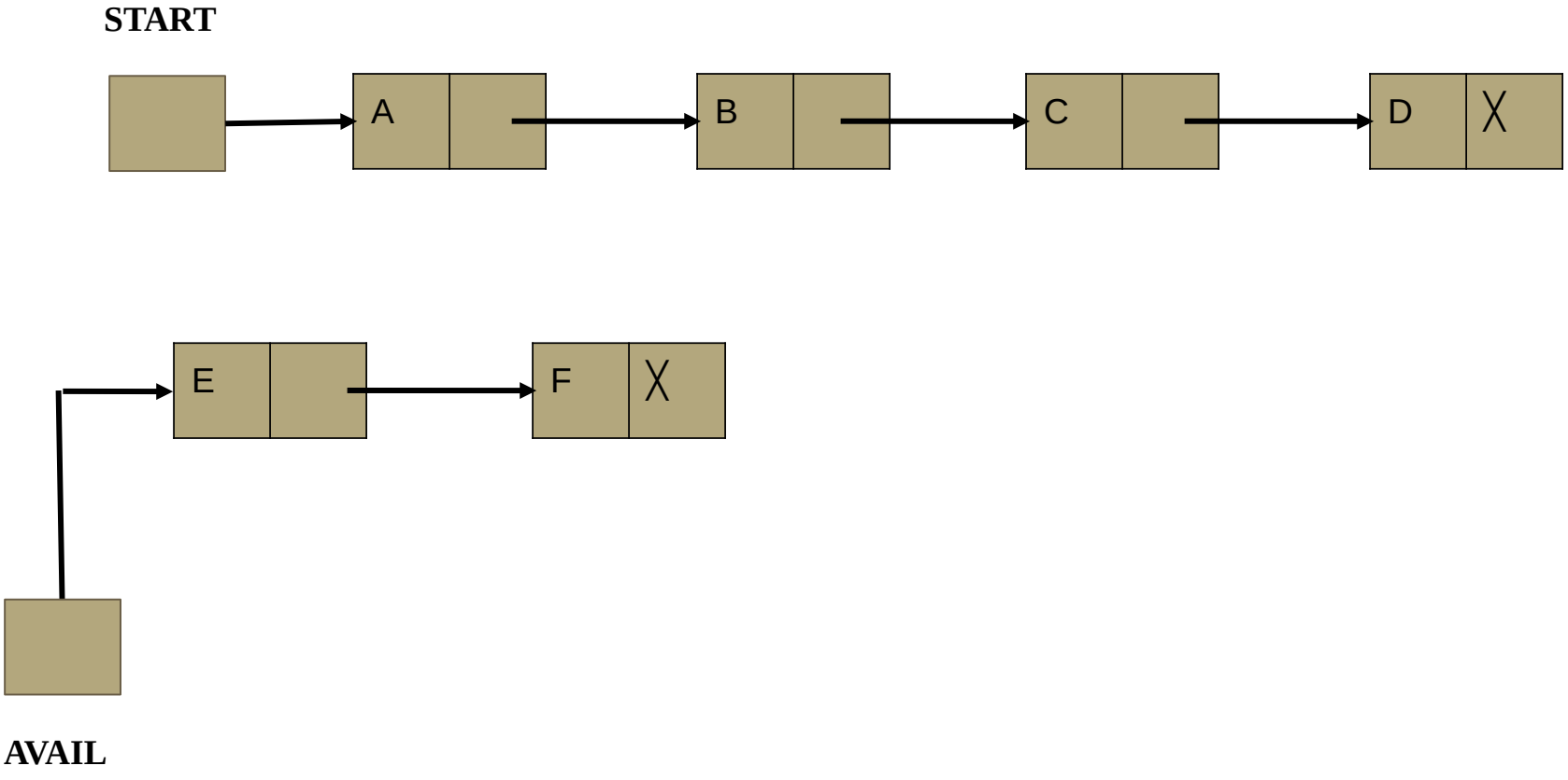


Insertion at the End in a Linked List

In the general case, traversal to the last node is required, making time complexity linear with the number of nodes.

- Worst- O(n)
- Average: O(n)
- Best: O(1)

Insertion After a given location in a Linked List



Insertion After a given location in a Linked List

1. **[Overflow?]**

   If AVAIL = NULL, print OVERFLOW and exit.

2. **[Remove first node from AVAIL list.]**

   Set NEW ← AVAIL

   Set AVAIL ← LINK[AVAIL]

3. **[Copy ITEM.]**

   Set INFO[NEW] ← ITEM

4. **[Check LOC and Insert:]**

   If LOC = NULL:

   Set LINK[NEW] ← START

   Set START ← NEW

   (Insert as first node)

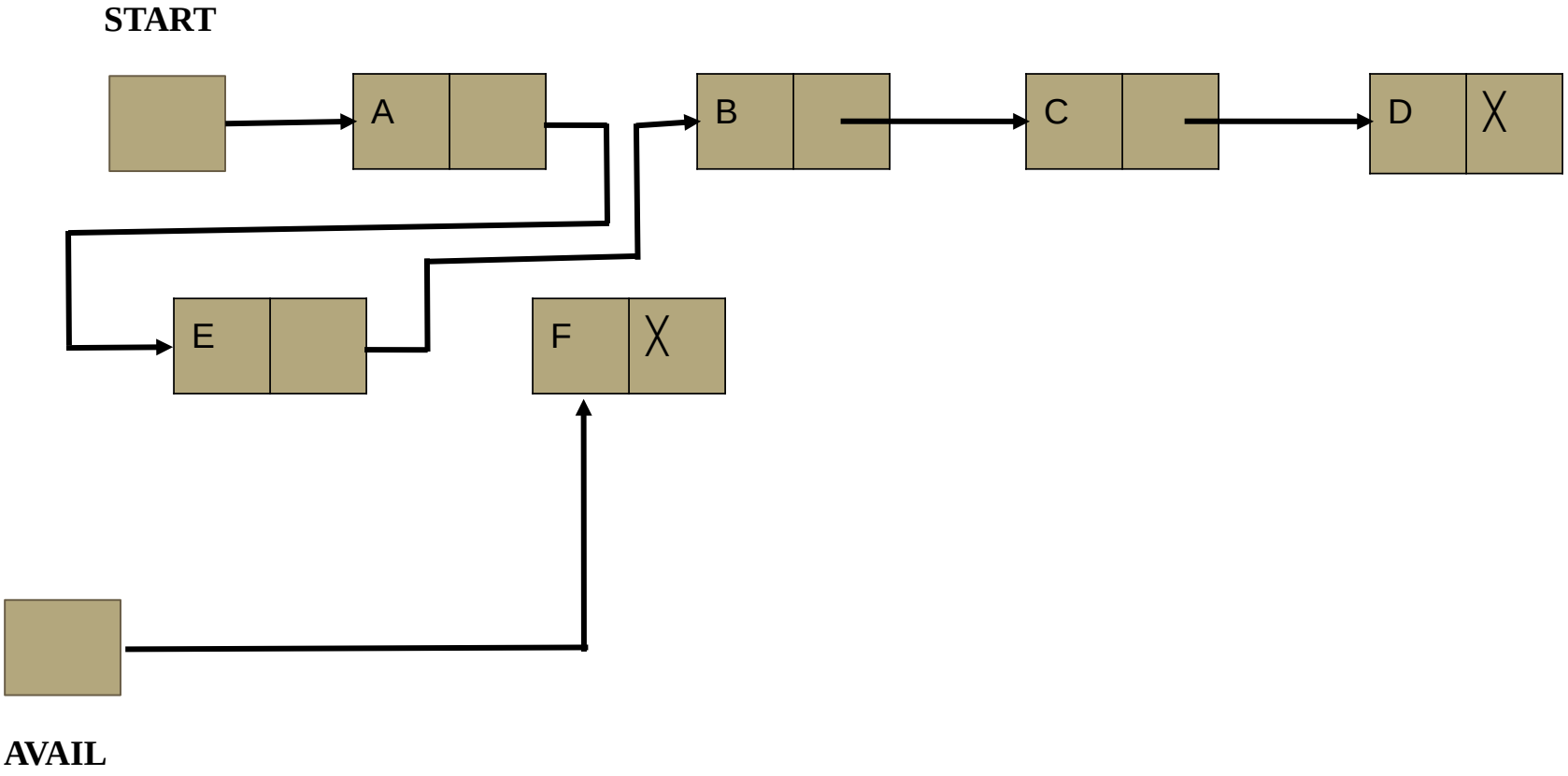
   Else:

   Set LINK[NEW] ← LINK[LOC]

   Set LINK[LOC] ← NEW

   (Insert after LOC)

Insertion After a given location in a Linked List



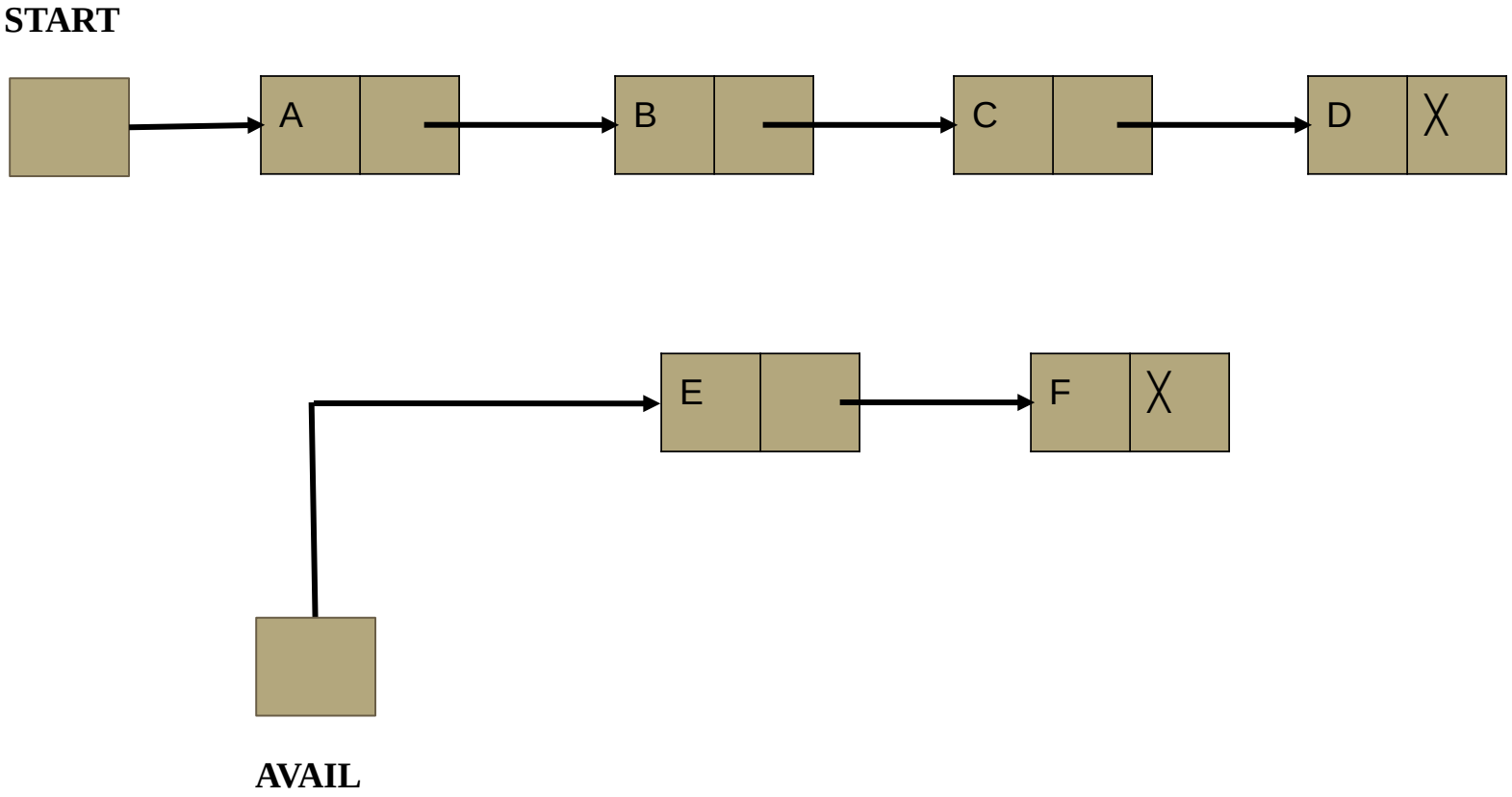
Insertion After a given location in a Linked List

- **Best Case:** O(1) (Insert at START directly)
- **Average Case:** O(1) (Given LOC is known)
- **Worst Case:** O(1) (Always constant time, as LOC is directly given)



Deletion from a Linked List

Deleting a node following a given Nod



Deleting a node following a given Node

Deletes node N with location LOC. LOCP is the location of the preceding node or NULL.

1. **[If deleting first node?]**

   If LOCP = NULL, then:

   Set START ← LINK[START]

   (Deletes first node.)

   Else:

   Set LINK[LOCP] ← LINK[LOC]

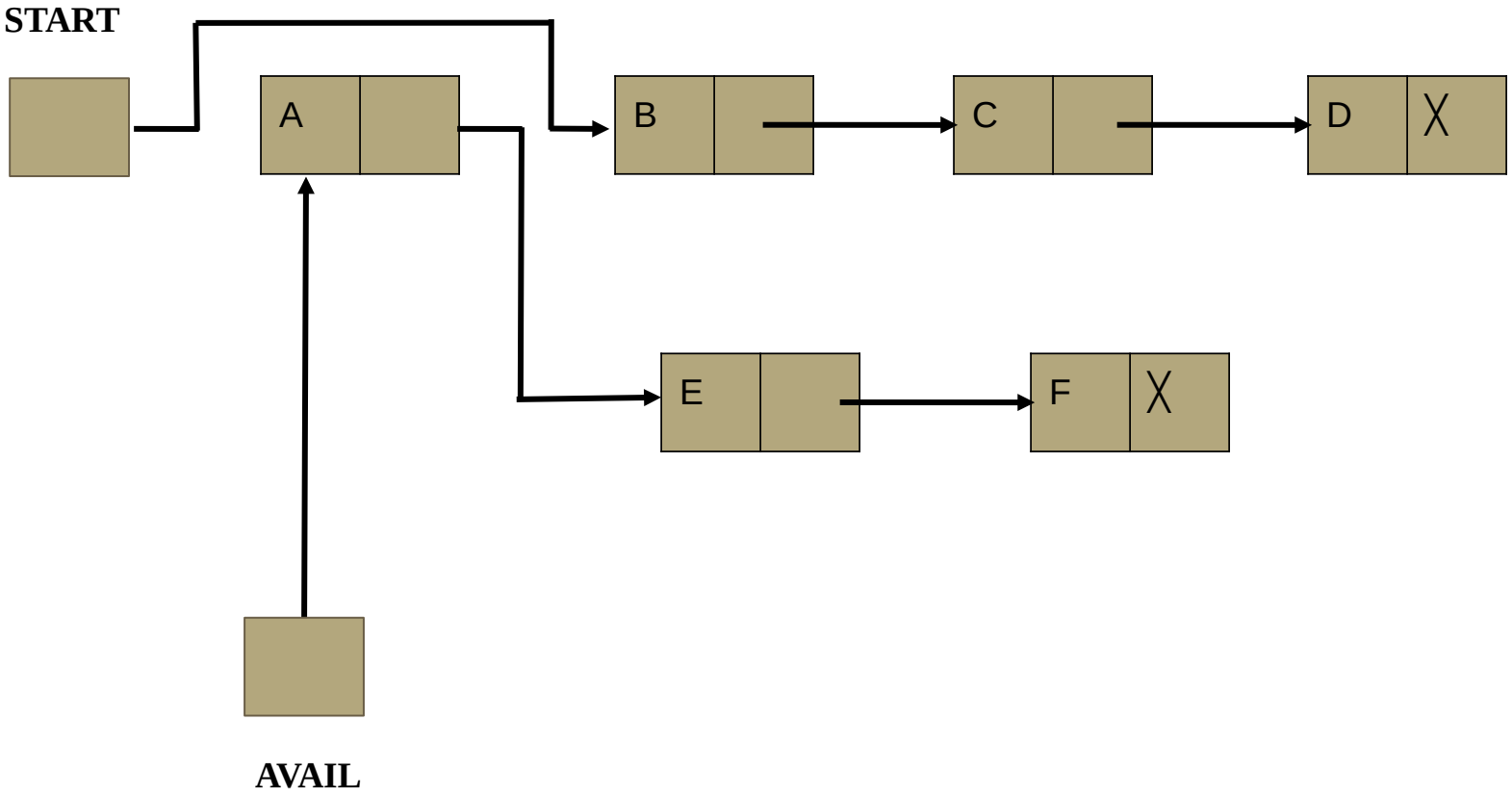
   (Deletes node N after LOCP.)

2. **[Return deleted node to AVAIL list.]**

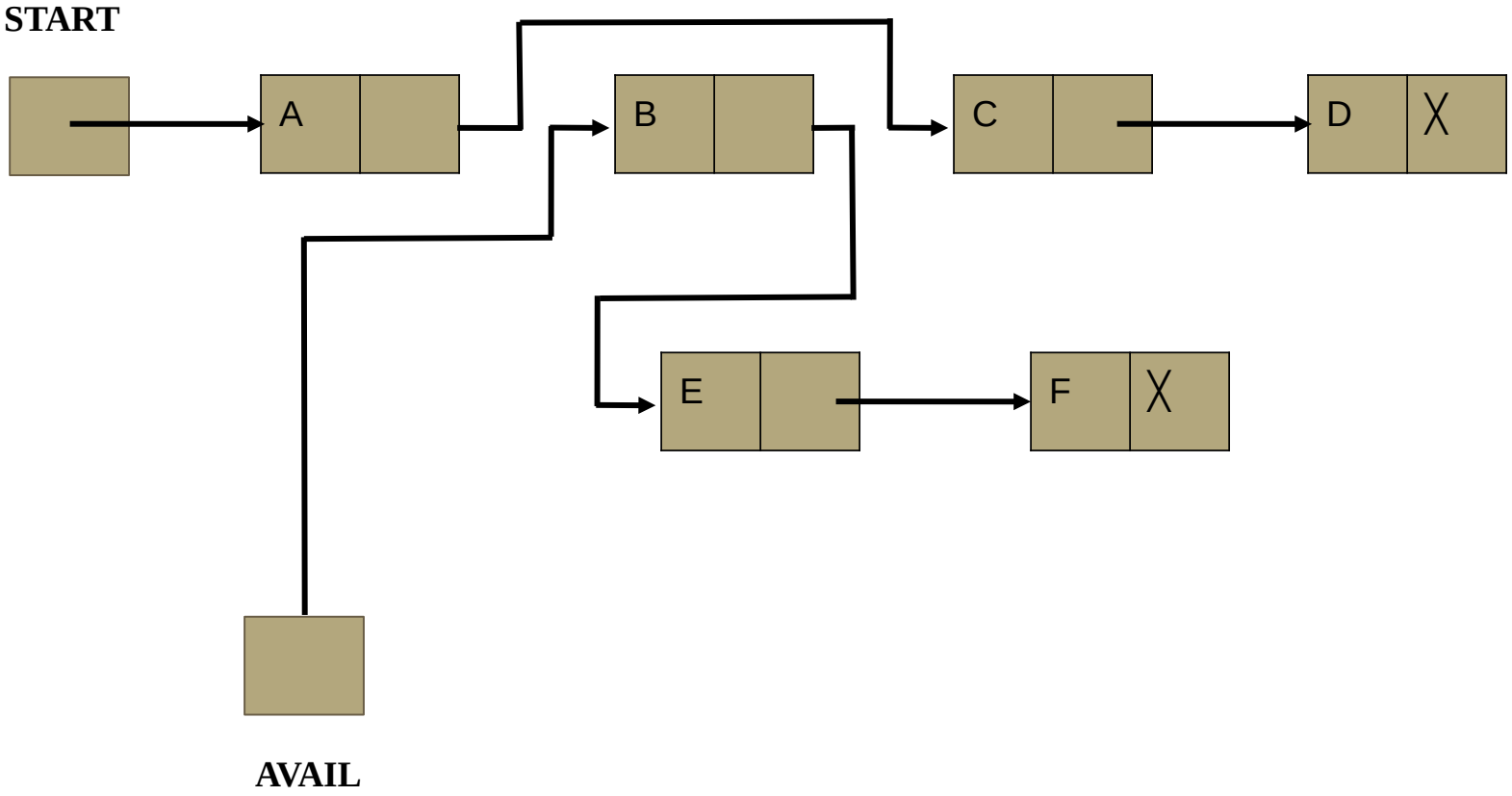
   Set LINK[LOC] ← AVAIL

   Set AVAIL ← LOC

1.Deletion from the Beginning



2. Deleting a node following a given Node

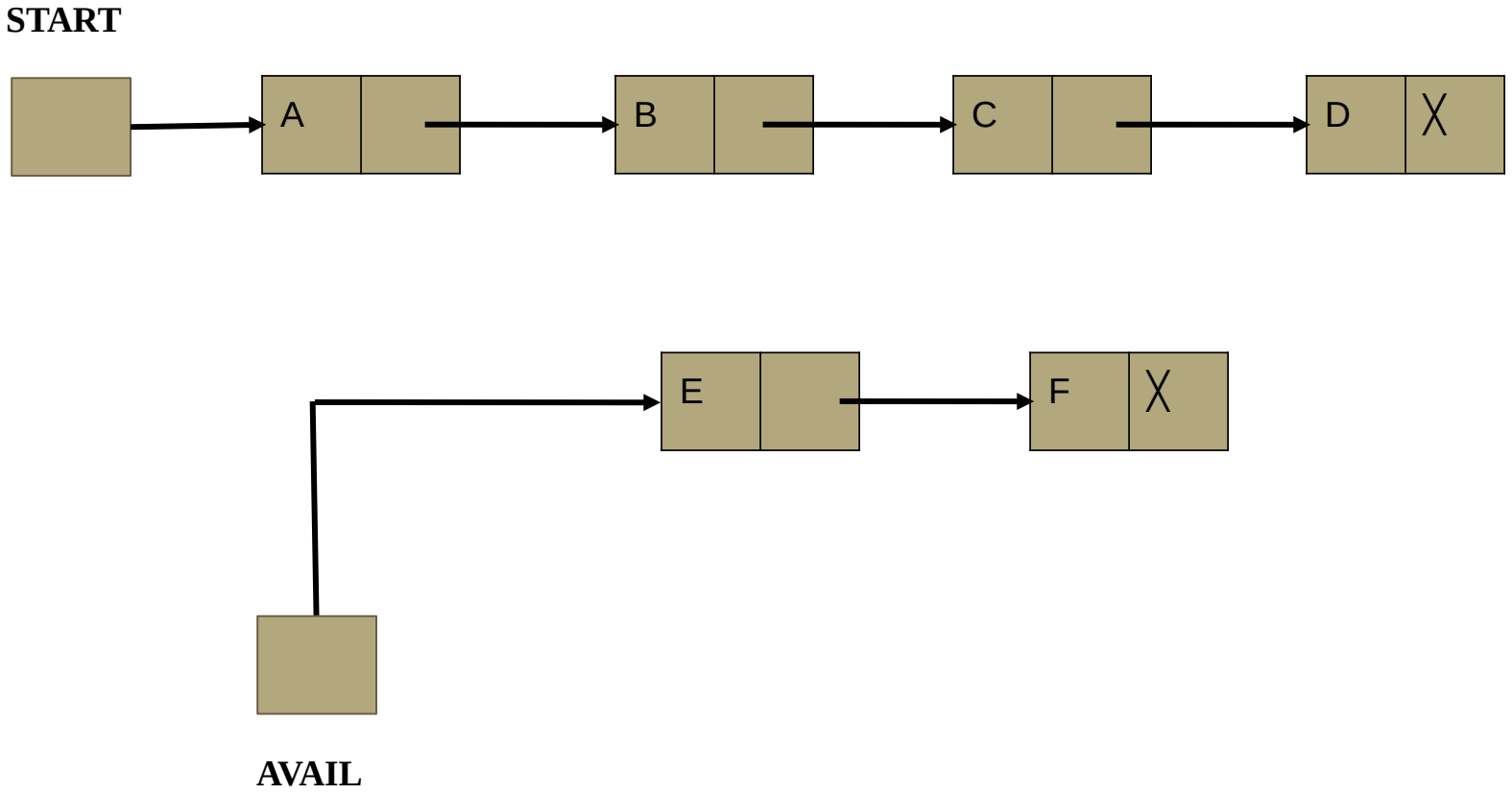


Deleting a node following a given Node

Since deletion only involves pointer updates; LOC and LOCP are assumed to be known.

- **Best Case:** O(1)
- **Average Case:** O(1)
- **Worst Case:** O(1)

Deleting a node with a given Information



Deleting a node with a given Information.

Find the first node (LOC) containing ITEM and its previous node (LOCP).

1. **[Check empty list]**
   If START = NULL:
   Set LOC ← NULL and LOCP ← NULL.
   Return.
2. [Check first node]
   If INFO[START] = ITEM:
   Set LOC ← START and LOCP ← NULL.
   Return.
3. [Initialize pointers]
   SAVE ← START
   PTR ← LINK[START]

4. **[Traverse the list]**
   Repeat while PTR ≠ NULL:
   If INFO[PTR] = ITEM:
   Set LOC ← PTR
   and LOCP ← SAVE.
   Return.
   Else:
   SAVE ← PTR
   PTR ← LINK[PTR]
5. **[ITEM not found]**
   Set LOC ← NULL.
   Return.

Deleting a node with a given Information.

Delete first node containing ITEM.

1. **Find target node:**
   Call FINDB(INFO, LINK, START, ITEM, LOC, LOCP)
2. **If ITEM not found:**
   If LOC = NULL:
   a. Output "ITEM not in list"
   b. Exit.

1. **Delete node N at LOC:**
   If LOCP = NULL (deleting first node):
   START ← LINK[START]
   Else:
   LINK[LOCP] ← LINK[LOC]

5. **Return deleted node to AVAIL:**
   LINK[LOC] ← AVAIL
   AVAIL ← LOC
6. **Exit**

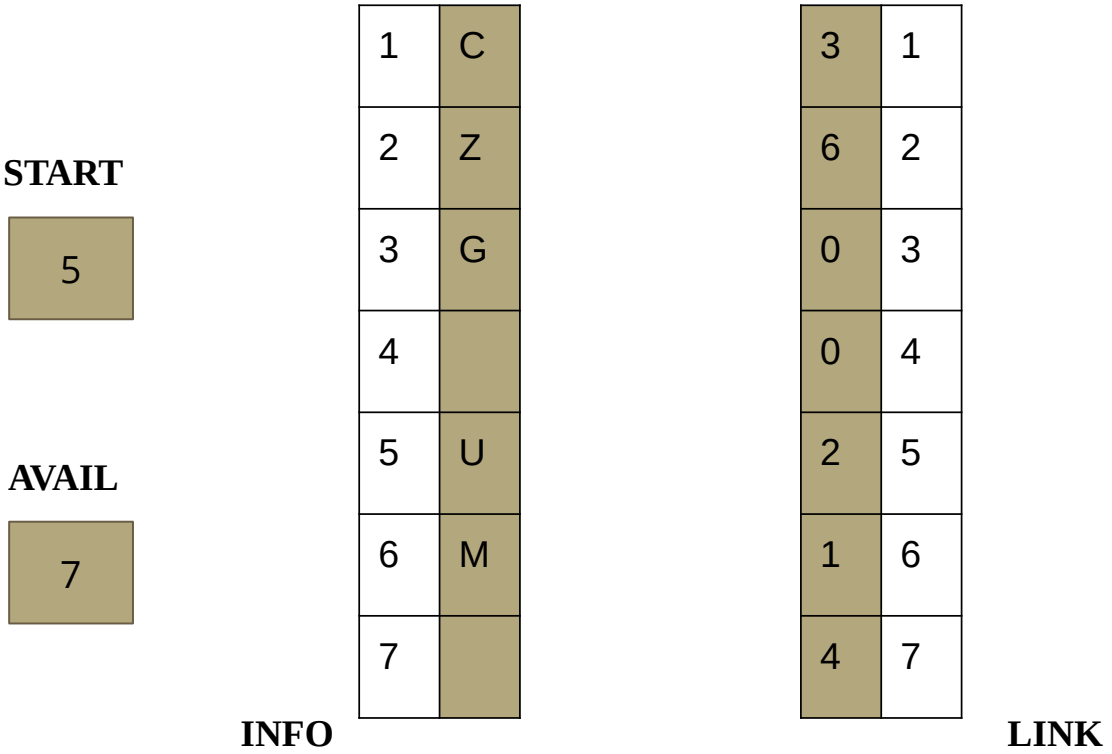
Deleting a node with a given Information

Time Complexity:

- **Best Case:** O(1) (If ITEM is in the first node.)
- **Average Case:** O(n)
- **Worst Case:** O(n) (If ITEM is at the end or absent.)

Exercise

(a) Find the sequence of characters in the list.



Exercise

Suppose Z and then C are deleted from the list and then H is inserted at the beginning of the list. Find the final structure.

START

5

AVAIL

7

	INFO	LINK
1	C	3
2	Z	6
3	G	0
4		0
5	U	2
6	M	1
7		4